

HAVEN

Rapport Technique Officiel — V3 (corrections post-bêta humaine V2)

Suite à une bêta humaine réelle de HAVEN V2, cette V3 corrige le déséquilibre économique révélé (stockage/citerne restaient inutiles), clarifie l'intitulé « Productivité », corrige un saut d'animation à vitesse maximale, rend les messages post-achat temporaires, et ajoute une direction visuelle de campagne. Aucune Map 2, aucune monétisation. Dépôt : [scuillerjack-oss/haven-v0](https://github.com/scuillerjack-oss/haven-v0) · Branche : main.

Convention : **FAIT** = codé. **TESTÉ** = vérifié par un test automatisé (moteur ou navigateur headless). **DÉPLOYÉ** = le workflow GitHub Actions a réussi sur le commit concerné. **NON VÉRIFIÉ** = pas confronté à un vrai appareil Android depuis cette session. Aucune simulation ni navigateur headless n'est présenté comme une validation Android humaine.

0. Audit Git réel avant toute modification

Vérifié par git status, git log et git fetch avant toute modification : arbre de travail propre, aucune modification non committée, HEAD local identique à origin/main (**08bac11**, dernier commit de la mission V2 — le rapport officiel V2). 48 tests node:test verts localement à ce commit. Dernier déploiement GitHub Actions (run #14) confirmé réussi via l'API GitHub, pas supposé.

1. Commits créés pendant cette mission V3

Commit	Jalon	Contenu
2c84a5e	1	Recalibre stockage/citerne pour une vraie alternance de goulots
705569c	2	Vérifie l'effet réel de « Productivité » et la renomme « Rendement de vente »
0fcd293	3	Découple le rendu de l'animation du tick économique (corrige le saut à vitesse max)
1f1b6d0	4	Messages post-achat transitoires (~3s)
4fd8802	5	Direction visuelle campagne (décor + pictogrammes + pièce)
a3da9bc	6	Recalibre la cible de fin de map après la réduction du plafond de la citerne

Chaque commit poussé séparément vers origin/main immédiatement après ses propres tests verts.

2. Déséquilibre économique révélé par la bêta (priorité absolue)

Cas de régression exact rapporté : déplacement, corde/treuil et puits déjà au niveau maximum, seau vers le niveau 7, alors que stockage et citerne pouvaient rester au niveau 1 sans jamais devenir limitants. Le diagnostic du jeu affichait alors en permanence « la production limite le débit ».

2.1 Cause racine (trouvée par simulation avec le vrai moteur, jamais supposée)

Le débit de transport de base (citerne 20 L / 60s en V2) était ~18 fois plus rapide que le débit de production de base. Pire : la citerne et le seau croissaient au même rythme relatif (même valueGrowth), donc cet écart de 18x se conservait à TOUT niveau — la production ne pouvait jamais rattraper le transport, quels que soient les achats. Même quand la citerne était achetée, elle atteignait déjà son niveau maximum bien avant que la production ne s'en approche.

2.2 Correctif : resserrer l'écart, jamais juste ralentir tout le monde

Recalibré pour que citerne ne soit plus qu'environ 3.5x plus rapide que la production au niveau 1 (au lieu de 18x), avec un nombre de niveaux et un coût qui l'égalent sur une durée comparable à celle du seau (au lieu d'être épuisée en une fraction du temps) :

Famille	V2 (avant)	V3 (après)
Citerne — niveaux	12	8
Citerne — capacité niveau 1	20 L	4 L

Famille	V2 (avant)	V3 (après)
Citerne — capacité niveau max	232.83 L	14.33 L
Citerne — coût niveau max	62 338	4 759
Stockage — capacité niveau 1	20 L	10 L
Stockage — capacité niveau max	232.83 L	62.75 L
Débit max théorique de la chaîne	~38.8 L/s (jamais transport-limité)	~2.4 L/s (transport devient limitant)

Effet secondaire détecté par la même exigence de simulation (jalon 6) : réduire ce plafond réduit aussi le débit maximal de toute la chaîne, ce qui aurait fait exploser la durée totale avec l'ancienne cible de 600 000 L (le profil léger ne terminait plus sous 10 jours). Recalibré à 120 000 L pour retrouver une durée proche de celle mesurée en V2 (section 5).

2.3 Résultat mesuré (scripts/simulate-v3-bottleneck.mjs, reproductible)

Séquence macro des goulots pendant une partie jouée gloutonnement (achète tout ce qui est affordable) : **production (0 min) → transport (36 min) → production (42 min) → transport (45 min) → stockage plein / transport en régime permanent**. Une vraie alternance lisible, plus un unique verrouillage sur « production » du début à la fin. Au moment précis où les 3 familles de vitesse sont toutes maxées ($t=36$ min), la citerne n'est encore qu'au niveau 5/8 (des niveaux réellement achetablement restent) et le goulot signalé est désormais « transport », jamais plus « production ».

Garde-fous permanents (tests/bottleneck.test.js) : le cas de régression exact ne doit plus jamais se reproduire ; la citerne ne doit jamais être épuisée avant d'avoir servi ; une vraie alternance doit se produire pendant la partie. Début rapide préservé (premier achat 24s, 8 achats en 3 min, tests/pacing.test.js inchangés et toujours verts).

3. « Productivité » vérifiée puis renommée « Rendement de vente »

Effet réel vérifié dans mapEconomy.js avant tout choix de libellé (jamais deviné) : ce multiplicateur n'augmente JAMAIS le débit physique (litres produits ou transportés) — il multiplie uniquement l'argent gagné par litre réellement vendu, uniformément sur toute la chaîne. D'où le nouveau nom et un affichage en grandeur concrète (« 2 → 2.3 par litre vendu ») plutôt qu'un pourcentage abstrait.

Bug trouvé au passage par cette même vérification : le calcul hors-ligne appliquait ce multiplicateur AVANT de calculer les litres transportés, contrairement au chemin actif qui ne l'applique qu'à l'argent. Un même achat aurait donc fait progresser la carte plus vite hors-ligne qu'en jeu actif — une incohérence jamais voulue, corrigée pour que les deux chemins soient identiques. Nouveau test permanent figeant cet invariant (« jamais plus de litres physiques, seulement plus d'argent », les deux chemins).

4. Saut d'animation à vitesse maximale : cause exacte et correctif

Diagnostic demandé explicitement (« ne pas se contenter des planchers mathématiques qui passent les tests, la validation qui compte est visuelle ») : le positionnement du travailleur était recalculé uniquement dans la boucle du tick économique, un `setInterval` à 200ms (`TICK_MS`). Une phase au plancher de vitesse visuel ne dure que 250-260ms — à peine plus d'une seule image de rendu à cette cadence, donc un déplacement qui saute au lieu de glisser. Aux vitesses normales du début de partie (phases de 1200-1800ms), 200ms donnait assez d'images intermédiaires pour paraître fluide ; le problème n'apparaissait qu'une fois les axes de vitesse proches de leur maximum — exactement comme rapporté.

Corrigé sans toucher à l'économie (le tick reste à 200ms, déterministe, inchangé) : le rendu de la position/marche du travailleur tourne maintenant sur `requestAnimationFrame` (cadence de l'écran, 60-120Hz), `currentProducerPhase()` acceptant un nouveau paramètre `extraMs` pour extrapoler visuellement le temps écoulé depuis le dernier tick — jamais au-delà de la fin de l'étape en cours, jamais une information mensongère sur l'état réel du moteur.

Mesuré en navigateur (Playwright, sauvegarde injectée avec déplacement/corde-treuil/puits déjà maxés) : 57 positions x distinctes échantillonnées sur 150 lectures en ~3s pendant la marche, contre un saut brutal en une poignée d'images auparavant.

5. Simulations économiques avant/après (valeurs mesurées, jamais estimées)

Profil	Map 1 complète — V2	Map 1 complète — V3	Actif — V2	Actif — V3
Léger (2×1min/j)	3.5 j	3 j	7 min	6 min
Modéré (3×3min/j)	2.33 j	2 j	21 min	18 min
Assidu (5×5min/j)	1.375 j	1 j 4 h	35 min	30 min

Durée totale proche de la V2 sur les 3 profils (légèrement plus courte, dans la marge attendue d'un recalibrage, jamais une explosion ni un effondrement). Début rapide inchangé : premier achat en 24s, 8 décisions cumulées en 3 minutes de jeu actif continu (identique à la V2, ces axes n'ont pas été touchés).

Reproductible : `node scripts/simulate-v2.mjs` (rythme de démarrage + durée totale) et `node scripts/simulate-v3-bottleneck.mjs` (alternance des goulots), résultats dans `docs/haven-v2-simulation-results.json` et `docs/haven-v3-bottleneck-simulation-results.json`.

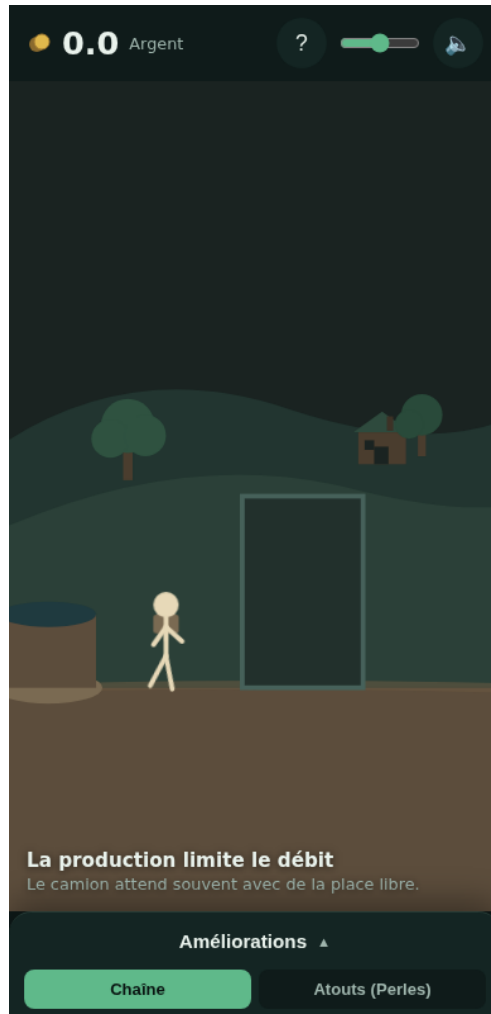
6. Messages contextuels après achat : feedback temporaire

Composant visé : l'onboarding contextuel (`showTutorialOnce/hideTutorial`), jusqu'ici affiché sans jamais se cacher tout seul (visible jusqu'à la prochaine action pertinente, ou indéfiniment). Corrigé sans changer QUAND ces messages se déclenchent : apparition immédiate, disparition douce après ~3s (fenêtre demandée : 2,5-3,5s) via une transition CSS `opacity` ; un nouveau message annule proprement le minuteur en cours et repart pour un cycle complet — jamais de chevauchement, jamais un ancien texte qui réapparaît après le nouveau.

4 tests permanents déterministes (`node:test mock.timers`, jamais un vrai délai attendu) figent la fenêtre de 3s, le remplacement propre, l'annulation par `hideTutorial`, et le comportement « une seule fois par partie » inchangé.

7. Direction visuelle : petit lieu de campagne

Décor statique ajouté en arrière-plan (collines, 2 arbres, cabane distante), un pictogramme SVG par famille d'amélioration, une pièce à côté de l'argent — formes plates légères (SVG/CSS inline), aucune image téléchargée, cohérentes avec la palette sombre/verte déjà en place. Jamais superposé aux éléments fonctionnels : positionné au-dessus de `y=260` (le sommet de la citerne de stockage, l'élément fonctionnel le plus haut — zone garantie libre quel que soit `x`) et entre `x=85` et `x=315` (visible même à la largeur la plus étroite testée, 320px) — vérifié par mesure réelle des rectangles rendus, pas supposé.



Capture d'écran réelle (Playwright, 390px de large) : scène avec le décor de campagne.

Simplification assumée, honnête, pas un défaut caché : formes géométriques plates (arbres/cabane stylisés), pas une illustration peinte comme la référence visuelle du cahier des charges — cohérent avec le style déjà minimaliste du stickman/puits. Une illustration peinte est hors de portée d'un agent de code sans pipeline d'art ; reproduire la DIRECTION (campagne, éléments identifiables, palette cohérente) plutôt que le rendu pixel-à-pixel, comme le cahier des charges l'autorise explicitement.

Nouvelle vérification automatisée permanente (check-mobile-resume.mjs) : le décor reste entièrement dans le cadre visible aux 5 largeurs déjà testées (320-480px), pas seulement à la largeur utilisée pendant le développement.

8. Périmètre respecté

- Aucune Map 2 construite (hors périmètre explicite de cette V3).
- Aucune interface de monétisation (hors périmètre explicite, décision de prudence maintenue).
- Renaissance/Perles/atouts non modifiés inutilement — seule la famille « Rendement de vente », propre à la Map 1, a été touchée (renommage + correction de cohérence, jamais une refonte).

9. Tests exécutés et résultats exacts

57 tests node:test (0 échec) + suite Playwright mobile (5/5 largeurs + décor visible, 0 échec) :

Fichier	Nb tests	Couvre
bottleneck.test.js	3	NOUVEAU V3 — cas de régression exact, citerne jamais épuisée avant de servir, vraie alternance
economy.test.js	15	Production, coûts, goulots, saturation, hors-ligne, NOUVEAU V3 : extraMs, rendement de vente
mapDefinitions.test.js	4	Planchers de vitesse visuelle, croissance stricte des coûts
meta.test.js	6	Perles, Renaissance, atouts (aucune régression, non modifiés)
migrations.test.js	7	Migration V0→V1→V2, champs absents
onboarding.test.js	5	NOUVEAU V3 — fenêtre de 3s, remplacement propre, annulation, une fois par partie
pacing.test.js	5	Premier achat ≤40s, plusieurs décisions en 3 min, durée totale non raccourcie
save.test.js	5	Sauvegarde/rechargement, aucune perte
simulation.test.js	3	Intégration tick + hors-ligne, déblocage de map
upgrades.test.js	5	Achats, refus, réduction de coût, conformité valeur affichée ↔ moteur

Suite mobile (npm run test:mobile) : clic fonctionnel après suspend/resume, aucune erreur console, crédit hors-ligne idempotent, aucun débordement horizontal, décor de campagne entièrement visible — les 5 vérifications, aux 5 largeurs (320/360/390/414/480px). Build de production propre : CSS 8.59kB (2.71kB gzip), JS 42.14kB (13.80kB gzip) — variation négligeable malgré le nouveau décor.

10. État réel du déploiement

7 push sur main pendant cette mission, chacun avec le workflow « Deploy to GitHub Pages » — vérifié via l'API GitHub Actions, pas supposé. État confirmé sur le dernier commit (**a3da9bc**, run #21) : job build réussi (npm ci, npm test, npm run build, playwright install, npm run test:mobile — chaque étape verte individuellement) ET job deploy réussi (actions/deploy-pages@v4). URL inchangée : <https://scuillerjack-oss.github.io/haven-v0/>.

Limite honnête, identique à celle déjà documentée en V1/V2 : cette session tourne dans un environnement dont la politique réseau bloque github.io (confirmé de nouveau par un test direct). Seule l'API GitHub confirme que le déploiement a réussi — un chargement réel sur téléphone reste la vérification qui compte et n'a pas été refait pour cette V3 précise : **NON VÉRIFIÉ**.

11. Limites connues et éléments non vérifiés

- **NON VÉRIFIÉ** — Les 4 correctifs de cette V3 (alternance de goulots, rendement de vente, fluidité de l'animation, messages temporaires) confirmés par simulation/Playwright headless, PAS encore par un nouveau test humain réel sur l'appareil Android où les problèmes ont été observés.
- **NON VÉRIFIÉ** — Décor de campagne jamais vu sur un écran Android réel (seulement Chromium headless à 5 largeurs simulées).
- Simplification assumée — décor géométrique plat, pas une illustration peinte (section 7).

- REPORTÉ — Map 2 et monétisation, hors périmètre explicite de cette V3.

12. Recommandations pour la suite

- Confronter les 4 correctifs de cette V3 à un vrai test humain sur le même appareil Android que les bêtas précédentes, avant toute nouvelle fonctionnalité.
- Si ce test confirme la boucle Map 1 comme saine, la prochaine évolution peut porter sur Renaissance/Perles et les maps suivantes, comme prévu par le cahier des charges V3 lui-même.
- Étudier les interfaces rewarded/premium seulement après ce test humain — pas avant.

Fin du rapport — HAVEN V3, dépôt scuillerjack-oss/haven-v0, branche main.